

Arama ve Graf Temelleri

Algoritmalar ve Veri Yapıları | Hafta: 5

Çalıştırmak için `Space` basın

Arama Algoritmalarına Giriş

- Veri yığınları içinde hedef bilgiyi bulma stratejileri.
- Performans kritiği: Zaman (Time) ve Alan (Space) karmaşıklığı.
- Veri organizasyonunun (sıralı vs sırasız) arama hızına etkisi.

Doğrusal Arama (Linear Search)

- Diziyi baştan sona tek tek tarayan en ilkel yöntemdir.
- Verinin sıralı olmasına ihtiyaç duymaz.
- En kötü durumda (Worst Case) tüm elemanlara bakılır.

Doğrusal Arama: Karmaşıklık Analizi

- Best Case: Hedef ilk elemanda ($O(1)$).
- Worst Case: Hedef sonda veya yok ($O(n)$).
- Average Case: Ortalama $n/2$ karşılaştırma ($O(n)$).

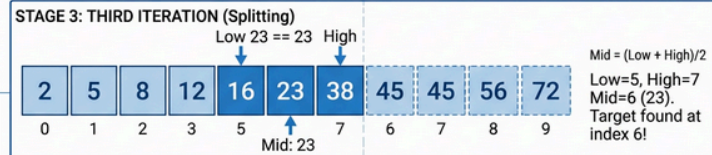
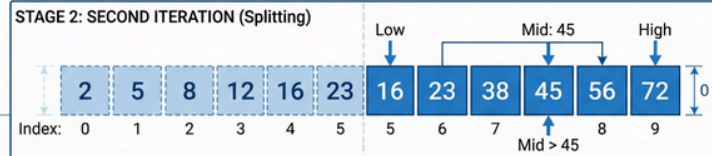
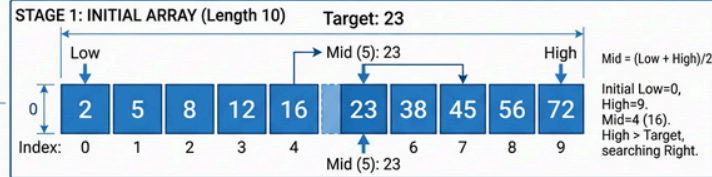
Doğrusal Arama Ne Zaman Kullanılır?

- Veri kümesi küçükse ($n < 50$).
- Veri tamamen düzensizse ve sıralama maliyeti yüksekse.
- Sadece bir kez arama yapılacaksa.

İkili Arama (Binary Search)

- Böl ve Yönet (Divide and Conquer) paradigmasıyla çalışır.
- Arama uzayını her adımda yarıya indirir.
- Logaritmik performans ($O(\log n)$) sağlar.

BINARY SEARCH VISUALIZATION: SORTED ARRAY (MINIMALIST BLUEPRINT)



Binary Search

Time Complexity: $O(\log n)$

Sıralı bir dizide aranan elemanı bulmak için arama aralığını her adımda yarıya indirir. Zaman karmaşıklığı $O(\log n)$ 'dir.

Status: **Ready**

6 8 16 30 33 36 45 54 56 76 78 85 92 94 99

Target Value:

76

Steps

0

binary_search.py

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```



▶ Play



Speed



Elements



Kritik Şart: Sıralı Veri

- Binary Search sadece **sıralı (sorted)** dizilerde çalışır.
- Sıralı veri, arama uzayını elemeye imkan tanır.
- Sıralı değilse, önce sıralama maliyeti ($O(n \log n)$) göze alınmalıdır.

İkili Arama: Adım Adım Mantık

- Dizinin orta elemanını (mid) belirle.
- Hedef == Mid ise, bulundu!
- Hedef < Mid ise, sol yarıyı tara (sağı çöpe at).
- Hedef > Mid ise, sağ yarıyı tara (solu çöpe at).

Binary Search: Özyinelemeli (Recursive)

- Problemi her seferinde kendisinin bir kopyasıyla ama daha küçük aralıkla çözer.
- Base Case: `low > high` ise bulunamadı.
- Kod okunabilirliği yüksektir.

Binary Search: Özyinelemeli (Recursive) (Algoritma)

```
1  binary_search(A, low, high, target):  
2      if low > high: return -1  
3      mid = low + (high - low) / 2  
4      if A[mid] == target: return mid  
5      if A[mid] > target: return binary_search(A, low, mid-1, target)  
6      return binary_search(A, mid+1, high, target)
```

Binary Search: İteratif (Döngüsel)

- Hafıza dostu: Sabit alan karmaşıklığı ($O(1)$).
- Stack Overflow riski taşımaz.
- Endüstriyel kodlamada genellikle bu yöntem tercih edilir.

Binary Search: İteratif (Döngüsel) (Kod Uygulaması)

```
1 def binary_search(arr, target):
2     low, high = 0, len(arr)-1
3     while low <= high:
4         mid = low + (high - low) // 2
5         if arr[mid] == target: return mid
6         if arr[mid] < target: low = mid + 1
7         else: high = mid - 1
8     return -1
```

Doğrusal vs İkili Arama Özeti

- Linear: 10^9 eleman $\rightarrow 10^9$ adım.
- Binary: 10^9 eleman $\rightarrow \log_2(10^9) \approx 30$ adım.
- Fark: Saniyeler vs Milisaniyeler.

Graf Teorisine Giriş

Bilgisayar bilimlerinde karmaşık ilişkileri ve sistemleri modellemenin matematiksel yoludur. Aşağıda bazı yaygın uygulamaların graflar üzerinden modelleme mantığını görebilirsiniz:

Uygulama	Eleman (Düğüm - Node)	Bağlantı (Kenar - Edge)
Harita ve Navigasyon	Kavşak, Şehir	Yol, Otoban
World Wide Web	İnternet Sayfası (URL)	Hiper Bağlayıcı (Link)
E-Ticaret ve Öneriler	Müşteri, Ürün	Alışveriş, Görüntüleme
Sosyal Ağlar	Kişi, Kullanıcı	Arkadaşlık, Takip

Matematiksel Tanım: $G = (V, E)$

- V (Vertices): Bağımsız nesneler veya noktalar.
- E (Edges): Nesneler arasındaki her türlü ilişki.
- İlişki varlığı bir küme eşleşmesi olarak ifade edilir.

Matematiksel Tanım ve Yönsüz Graflar

Bir Graf G , düğümlerden (Vertices - V) ve kenarlardan (Edges - E) oluşur:

$$G = (V, E)$$

- **Yönsüz Graf (Undirected):**

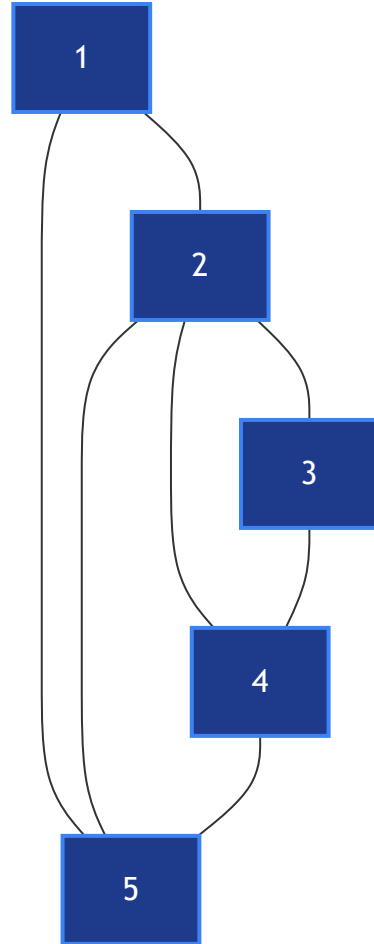
İlişki karşılıklıdır, yön belirtilmez. (Örn: 1 ile 2 bağlıysa 2 de 1 ile bağlıdır).

Undirected: İlişki karşılıklıdır (Örn: Facebook arkadaşlığı).

Kümelerle İfade:

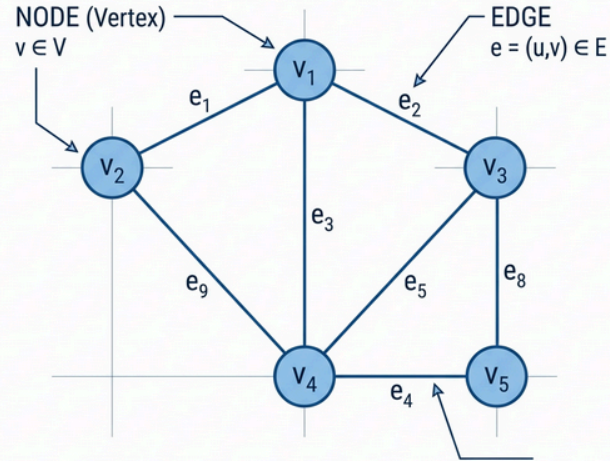
$$V := \{1, 2, 3, 4, 5\}$$

$$E := \{\{1, 2\}, \{1, 5\}, \{2, 5\}, \{2, 3\}, \{2, 4\}, \{3, 4\}, \{4, 5\}\}$$



GRAPH $G=(V, E)$ | BASIC ELEMENTS

VERTEX SET (V) | EDGE SET (E)



VERTEX SET $V = \{v_1, v_2, v_3, v_4, v_5\}$ (total $|V|=5$)

EDGE SET $E = \{(v_1, v_2), (v_1, v_3), (v_1, v_4), (v_3, v_5), (v_4, v_5), (v_3, v_4), \dots, (v_2, v_4)\}$
(total $|E|=9$)

Graf Terminolojisi: Derece (Degree)

- Degree: Bir düğüme bağlı olan kenar sayısıdır.
- In-Degree: Düğüme gelen oklar (Directed).
- Out-Degree: Düğümden çıkan oklar (Directed).
- Popülerlik veya kritiklik ölçütü olarak kullanılır.

Yönlü Graflar (Directed Graphs - Digraph)

İlişkinin tek yönlü akabildiği graflardır. (Örn: Twitter takibi, web sayfası linkleri).

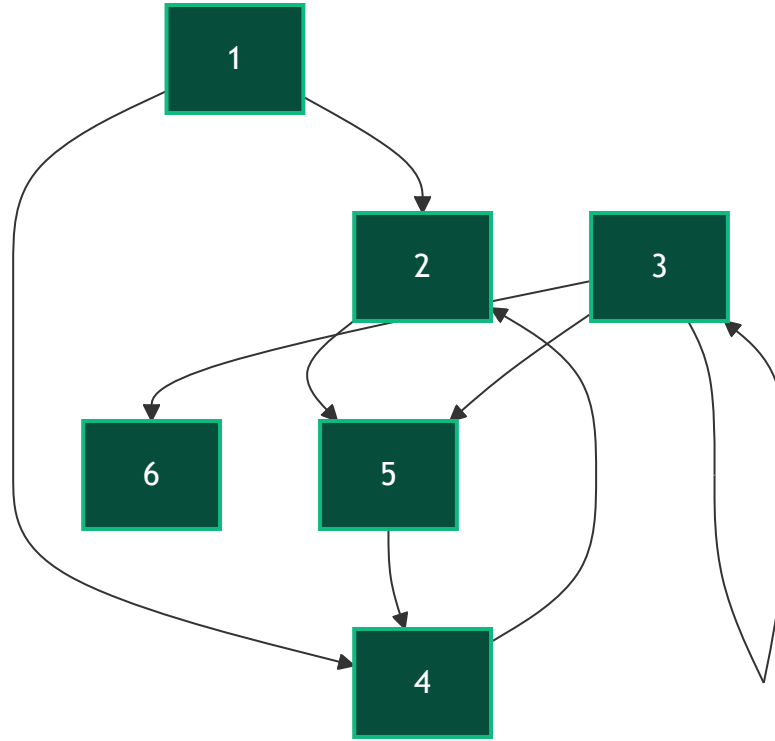
$$V := \{1, 2, 3, 4, 5, 6\}$$

$$E := \{(1, 2), (1, 4), (2, 5), (4, 2), (5, 4), (3, 3), (3, 5), (3, 6)\}$$

- (u, v) gösterimi kullanılır, yön u 'dan v 'ye doğrudur.
- **Teorem (Derece Toplamı):**

$$\text{Yönsüz için: } \sum \deg(v) = 2E$$

$$\text{Yönlü için: } \sum \text{indeg}(v) = \sum \text{outdeg}(v) = E$$

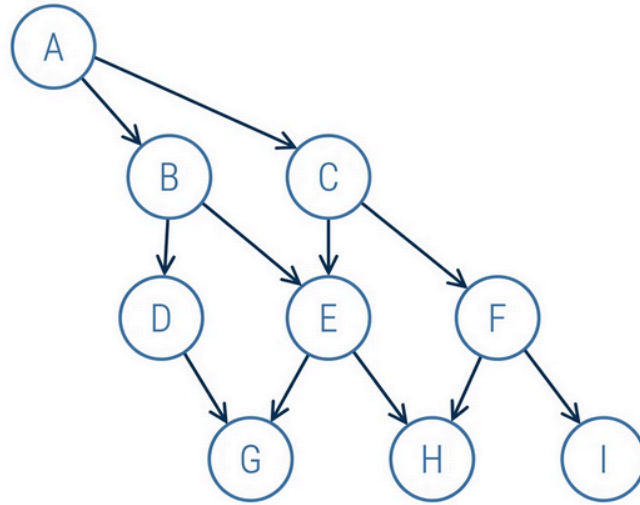


(Not: 3 numaralı düğümün 'kendine döngüsü' (self loop) vardır.)

Özel Bir Yapı: DAG

- Directed Acyclic Graph (Yönlü Döngüsüz Graf).
- İçinde döngü yoktur, bir akış her zaman ilerler.
- İş akışları ve bağımlılık haritaları (Build system) DAG'dır.

DIRECTED ACYCLIC GRAPH (DAG) VISUALIZATION



Educational Diagram: Structured, Acyclic Flow

Ağırlıklı Graflar (Weighted)

- Kenarlara sayısal bir değer atanmasıdır.
- Ağırlık: Mesafe, Maliyet, Kapasite veya Gecikme olabilir.
- Navigasyon sistemlerinde 'en kısa yol' hesabı bu ağırlıklarla yapılır.

Önemli Graf Terminolojileri

- **Paralel Kenarlar:** Aynı düğüm çiftini birbirine bağlayan birden fazla kenar (Çok şeritli yol gibi).
- **Kendine Döngü (Self Loop):** Bir düğümü kendine bağlayan kenardır.
- **Yol (Path):** Düğümler arası geçiş dizisidir. (Örn: A-B-C-D)
- **Basit Yol (Simple Path):** Hiçbir düğümün tekrar *etmediği* geçiş dizisidir.
- **Döngü (Cycle):** Başladığı noktada biten kapalı yoldur. Yazılımlarda tehlikeli (sonsuz döngü) veya finansal sahtecilik analizinde (para aklama zinciri) kâr getiren bir keşif olabilir.
- **Bağlılık (Connectivity):** Her düğümden her düğüme ulaşmanın mümkün olduğu, kopuk bir 'ada' (Component) barındırmayan sistemlerdir.

Ağaçlar (Trees): Kurallı Graflar

Ağaçlar, aslında çok katı kuralları olan birer graftır. Bir grafın ağaç olabilmesi için şu özelliklere sahip olması gerekir:

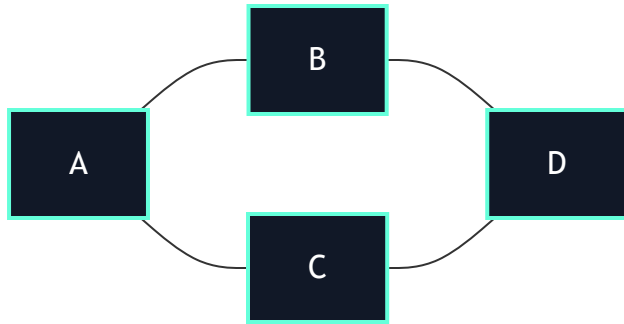
- Tamamen **Bağlı (Connected)** olmalıdır.
- İçinde hiçbir **Döngü (Acyclic)** barındırmamalıdır.
- Düğüm sayısı V ise, tam olarak $V - 1$ **adet kenarı** olmalıdır.
- Herhangi iki düğüm arasında sadece tek bir *basit yol* vardır.
- **Orman (Forest)**: Ağaçların bir araya gelerek oluşturduğu kümedir (bağlantısız ağaçlar bütünü).

Özel Graflar: Bipartite ve Komple

- **Bipartite Graflar (İki Kümeli):** Düğümleri kırmızı ve mavi olarak ayırdığımızda, hiçbir kırmızı bir başka kırmızıyla (veya mavi maviyle) bağlanamaz. İş-İşçi, Öğrenci-Ders atamaları için mükemmel modeldir.
- **Complete Graflar (Tam Graf):** Herkesin herkesle doğrudan komşu (bağlı) olduğu ütöpik graflardır. V düğümlü tam grafta kenar sayısı maksimumdur: $\frac{V(V-1)}{2}$

Graf Temsili: Komşuluk Matrisi (Adjacency Matrix)

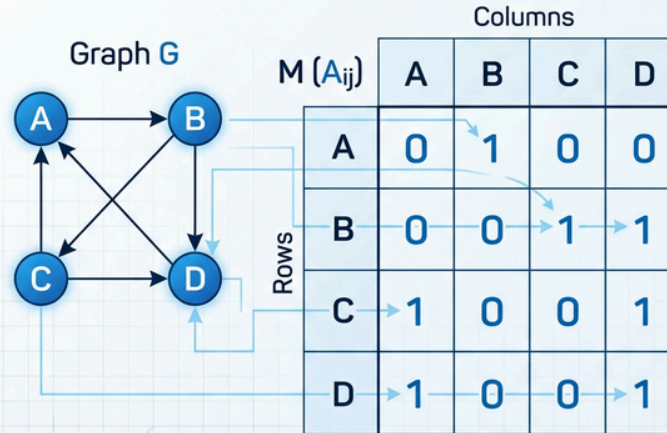
Grafı hafızada tutmanın en direk yolu 2-Boyutlu Array kullanmaktır. Boyutlar: $V \times V$. Kenar varsa 1 (veya ağırlık), yoksa 0 yazılır.



	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
<i>A</i>	0	1	1	0
<i>B</i>	1	0	0	1
<i>C</i>	1	0	0	1
<i>D</i>	0	1	1	0

Karmaşıklık: $O(V^2)$ bellek. Kenar sorgusu: $O(1)$.

ADJACENCY MATRIX REPRESENTATION OF A DIRECTED GRAPH (G)



$M[i, j] = 1$ if an edge exists from
Node i to Node j; 0 otherwise.

Ara Soru: Matris Maliyeti

1 Milyon (10^6) kullanıcısı olan bir sosyal ağı **Komşuluk Matrisi** ile RAM'de tutmak istiyoruz. Her bir hücrenin değeri için sadece 1 Byte (8 bit) harcarsek, matrisin boyutu toplamda yaklaşık kaç Terabyte (TB) olur?

Çözüm Adımları:

1. $V = 1.000.000$ düğüm.
2. Matris hücre sayısı $= V \times V = 10^{12}$ hücre.
3. Her hücre 1 Byte ise toplam $= 10^{12}$ Byte.
4. 10^{12} Byte $\approx$ 1 Terabyte (TB) sadece yapıyı tutmak için! (İçinde "1" olmayan %99 boşluk).

Kazanım: Seyrek (Sparse) graflarda matris kullanımının hafıza iflasına yol açacağını ve matematiksel hesaplama kapasitesinin kavrılması.

Soru: Matris Maliyeti ve Hafıza İflası (1/2)

1 Milyon (10^6) kullanıcısı olan bir sosyal ağı **Komşuluk Matrisi** ile RAM'de tutmak istiyoruz. Her bir hücre için sadece **1 Byte** harcarsak, bu matrisin boyutu yaklaşık kaç Terabyte (TB) olur?

Hücre Sayısının Tespiti:

Hesap 1

Matris $V \times V$ boyutunda olduğu için:

$$10^6 \times 10^6 = 10^{12} \text{ (1 Trilyon) hücre.}$$

Toplam Byte Miktarı:

Hesap 2

Her hücre 1 Byte yer kapladığından:

$$10^{12} \times 1 \text{ Byte} = 10^{12} \text{ Byte}$$

Soru: Birim Dönüşümü ve Analiz (2/2)

Birimlerin Karşılaştırılması:

10^3 Byte = 1 KB

Dönüşüm

10^6 Byte = 1 MB

10^9 Byte = 1 GB

10^{12} Byte = 1 Terabyte (TB)

Sonuç

Final: Yaklaşık 1 TB RAM gereklidir!

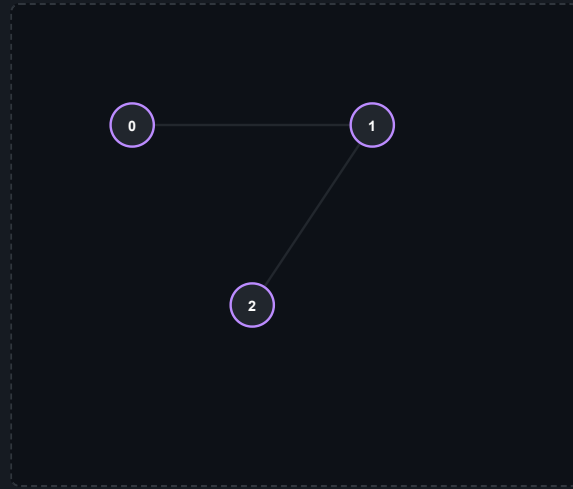
Kazanım (Analiz): Sosyal ağlarda bir kişi genellikle en fazla birkaç bin kişiyle bağlantılıdır. Bu durumda matrisin %99.9'u sıfır (boş) olacaktır. 1 TB'lık bu alanın neredeyse tamamı "hiçlik" tutmak için israf edilir. İşte bu yüzden **Seyrek (Sparse) Graflar** için her zaman liste yapısı tercih edilir.

Graph Representations

Nodes: 3 Edges: 2

Graf yapılarını görselleştirin ve Adjacency Matrix ile Adjacency List arasındaki farkı inceleyin.

Click to add node, drag to move, shift+drag between nodes to add edge. Clear



Adjacency Matrix

Adjacency List

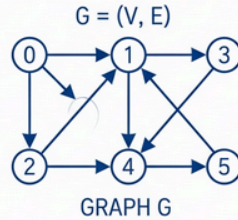
	0	1	2
0	0	1	0
1	1	0	1
2	0	1	0

Graf Temsili: Komşuluk Listesi (Adjacency List)

Her düğüm için, sadece o düğüme **bağlı olan (komşusu olan)** düğümlerin listesini tutma stratejisidir.

- $A \rightarrow [B, C]$
- $B \rightarrow [A, D]$
- $C \rightarrow [A, D]$
- $D \rightarrow [B, C]$

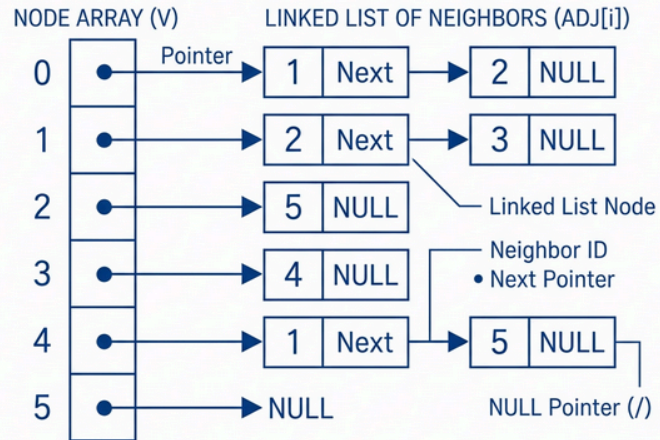
Karmaşıklık: $O(V + E)$ bellek harcar. "Kim komşu?" sorusunu satırı tarayarak $O(deg(V))$ sürede çözer. Seyrek (Sparse) graflar için en çok tercih edilen tiptir.



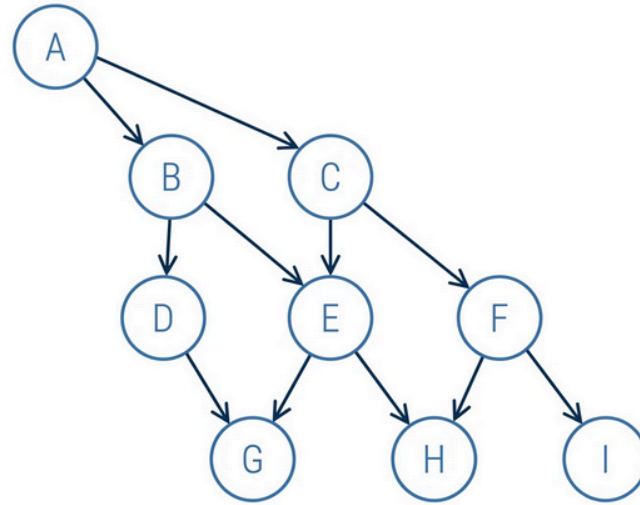
ADJACENCY LIST REPRESENTATION

- Graph G
- Vertices (Nodes)
 - Edges (Directed)

ADJACENCY LIST REPRESENTATION



DIRECTED ACYCLIC GRAPH (DAG) VISUALIZATION



Educational Diagram: Structured, Acyclic Flow

Diğer Modeller: Kenar Listesi ve Çapraz

- **Kenar Listesi (Edge List):** Tüm grafi sadece bir dizi veya tablo satırı gibi (Başlangıç, Bitiş, Ağırlık) tuple'ları halinde ardışık saklamak. Özellikle Kruskal gibi graf kenarlarını ağırlığına göre 'sıralayan' algoritmalar için muazzamdır.
 - Örn: (A, B, 15), (B, C, 22)
- **Çapraz Bağlantılı Listeler (Incidence List):** Her düğüm ve her kenar için ayrı nesne (class) oluşturulup obje referanslarıyla birbirlerine bağlanmasıdır. Nesne Yönelimli Programlamada (OOP) esneklik sağlar ancak bellek (pointer vs.) tüketimi daha büyüktür.

Graf Temsilleri: Karşılaştırma Matrisi

Hangi yapı, hangi durumda daha avantajlıdır?

İşlem	Komşuluk Matrisi	Komşuluk Listesi	Kenar Listesi	Çapraz Bağlantılı
Kurulum Alanı (Bellek)	$O(V^2)$	$O(V + E)$	$O(E)$	$O(V + E)$
Kenar (u,v) Var Mı?	$O(1)$	$O(deg(v))$	$O(E)$	$O(\min(deg(u), deg(v)))$
Tüm Komşuları Bulma	$O(V)$	$O(deg(v))$	$O(E)$	$O(deg(V))$
Kenar Ekleme	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Uygun Olduğu Senaryo	Yoğun (Dense)	Seyrek (Sparse)	Çok Seyrek / Kenar Sıralı	Karma / OOP

Hafta 5 Özeti ve Gelecek Hafta

- Sıralı verinin Binary Search ile $O(\log n)$ büyüklüğündeki operasyonel gücünü gördük.
- Gerçek dünya sistemlerini modellemek için Graf kavramını ve Yönlü/Yönsüz yapıları öğrendik.
- Hafıza/Hız denklemlerinde Matris ve Liste gösterimlerinin matematiksel maliyetlerini çıkardık.
- **Gelecek Hafta:** Kurduğumuz bu düğüm ve kenarlar dünyasında labirentleri nasıl çözeceğimizi, BFS (Arama) algoritmalarını işleyeceğiz.

Konu Sonu Soruları

Soru: Binary Search Performans Analizi (1/2)

1.000.000 elemanlı sıralı bir dizide **Binary Search** (İkili Arama) algoritması hedef elemanı bulmak için **en fazla** kaç karşılaştırma yapar?

Zaman Karmaşıklığı:

Teori

Binary Search, her adımda arama uzayını yarıya indirdiği için **Logaritmik Zaman Karmaşıklığına** ($O(\log_2 n)$) sahiptir.

Karşılaştırma Sayısı:

Mantık

Maksimum karşılaştırma sayısı, eleman sayısını 1'e indirene kadar kaç kez 2'ye bölebileceğimize eşittir:

$$Steps \approx \log_2(1.000.000)$$

Soru: Üstel Büyüme ve Karar (2/2)

2'nin Kuvvetleri ile Yaklaşım:

$$2^{10} = 1.024$$

Hesap

$$2^{19} \approx 524.288$$

$$2^{20} \approx \mathbf{1.048.576}$$

Final

Sonuç: En fazla 20 karşılaştırma yapılır.

Kazanım: Doğrusal aramada (Linear Search) 1 milyon adım sürebilecek bir işlem, Binary Search ile sadece 20 adımda biter. Bu, verinin büyüklüğüne rağmen algoritmanın hızı korumasını sağlayan logaritmik verimliliğin en net kanıtıdır.

Soru: Binary Search ve Aritmetik Taşma (1/2)

Binary Search algoritmasında orta noktayı bulurken $mid = (low + high) / 2$ yerine neden $mid = low + (high - low) / 2$ yazılması önerilir?

Integer Overflow (Tam Sayı Taşması):

Sorun

Eğer `low` ve `high` değerleri veri tipinin (örneğin 32-bit işaretli tamsayı) üst sınırına yakınsa ($> 2^{30}$), $(low + high)$ işlemi bellekte ayrılan sınırı aşarak **negatif** bir sonuç üretir.

Program Çöküşü:

Etki

Taşma sonucu oluşan negatif değer `2`'ye bölündüğünde, ortaya çıkan negatif indeks (`ArrayIndexOutOfBoundsException`) programın beklenmedik şekilde durmasına yol açar.

Soru: Güvenli Tasarım ve İspat (2/2)

Matematiksel Eşdeğerlik:

Çözüm

$$low + \frac{high - low}{2} = \frac{2 \cdot low + high - low}{2} = \frac{low + high}{2}$$

Formül matematiksel olarak aynıdır, ancak işlem sırasında `low + high` toplamı **hiçbir zaman gerçekleşmez**.

Fark

Önce fark (`high - low`) hesaplandığı için sonuç her zaman küçülür, ardından `low` üzerine ekleme yapıldığında toplam değer asla `high` değerini aşmaz. Bu, süreci güvenli aralıkta tutar.

Best-Practice: Bu durum, 20 yıl boyunca Java'nın standart kütüphanesinde gizli kalmış meşhur bir hatadır. Modern sistemlerde büyük veri setleri ile çalışırken bu "güvenli hesaplama" yöntemi zorunluluktur.